NATURAL LANGUAGE PROCESSING

Course Project Report

Predicting Story Points of Later Issues in a New Project using SBERT Sentence Embeddings

Problem Statement 8 (PS-8): Sentence embeddings (SBERT) · Cross-project · Temporal ordering

Submitted by **Group 1**

#	Name	Roll Number
1	Amal Faris M	B230162CS
2	Richie James	B230519CS
3	Vedavyas S Poika	B230607CS
4	Muhammed Ismayil	B230434CS
5	Janardhan Abhiram Bojja	B230979CS

GitHub repository: https://github.com/abhi230205/PS8-SBERT-StoryPoint-Estimation

Dataset: TAWOS (https://doi.org/10.5522/04/21308124)

November 2026

Abstract

Agile teams give every task a *story point* value that shows how much effort it needs. In this project we tried to predict story points automatically from the text of a task. We used **SBERT** to turn each sentence of the task into a vector, and a small **three-layer CNN** to predict the story points from these vectors. The model is trained on one software project and tested on the **most recent issues of a different project**, for all 39 project pairs given in the guidelines.

On the held-out test sets the model gets a mean absolute error (MAE) of **2.21** and a median absolute error (MdAE) of **1.85** story points. This is almost the same as a simple baseline that always predicts the median story point of the training project (MAE 2.16, MdAE 1.88). The model does beat this baseline inside its own training project, so it learns something from the text. That knowledge, however, does not carry over well to a new project, mainly because every team uses its own story point scale.

Contents

1. Introduction
2. Background
3. Dataset
4. Methodology
5. Results
6. Discussion: best representation, explanations, limitations and threats to validity
7. Conclusion and what we learned
8. How to run the code

References · Appendix A: results for every project pair

1. Introduction

In agile software development, work is split into small tasks called **issues** (or user stories). Before the team starts working, it estimates the effort of each issue in **story points**, a relative number such as 1, 2, 3, 5 or 8. Good estimates help with planning, but they take time and depend on experience. Machine learning can help by predicting the story points directly from the issue's title and description. For example, from the text *"Fix the login failure when a user enters an incorrect password"* the model should output a number like 2.

A model is most useful for a **new project** that does not have much history yet. Such a project has to use a model trained on *another* project. Also, in real life we always predict *future* issues using *past* data. Our problem statement combines both of these ideas.

> **Problem statement (PS-8):** Evaluate SBERT-based sentence embeddings for predicting story points of later issues in a target project using chronological ordering.

Our objectives were to:

1. Represent each issue using SBERT sentence embeddings.

2. Train a three-layer CNN on a *source* project and test it on the latest issues of a different *target* project, for all 39 project pairs given in the guidelines.
3. Choose settings using five-fold validation on the training data only, and use the test data only once at the end.
4. Report MAE and MdAE, compare with simple baselines, and discuss the results and their limitations.

2. Background

2.1 Sentence embeddings and SBERT

A computer cannot use text directly, so the text must be turned into numbers, called an **embedding**. Word embeddings (for example from BERT) give one vector per word. **Sentence embeddings** give one vector for a whole sentence. **SBERT** (Sentence-BERT, Reimers & Gurevych, 2019) is a BERT model trained so that sentences with a similar meaning get similar vectors. We used the pre-trained SBERT model `all-mpnet-base-v2`, which turns any sentence into a vector of 768 numbers.

2.2 Cross-project and chronological evaluation

- **Cross-project:** the model is trained on one project (the *source*) and tested on another project (the *target*).
- **Chronological (temporal) ordering:** issues are sorted by their creation date, and the test set contains the *latest* issues. This is more realistic than a random split, where the model could learn from issues created after the ones it is tested on.

2.3 Evaluation metrics

If y is the true story point and $\hat{y}$ the prediction, then **MAE = mean(|y − ŷ|)** and **MdAE = median(|y − ŷ|)**. Both are measured in story points, and lower is better. MdAE is less affected by a few very wrong predictions. We also compare against two simple **baselines** that do not read the text at all: always predicting the **median**, or the **mean**, story point of the training project. A useful model should beat these baselines.

3. Dataset

We used the **TAWOS** dataset (Tawosi et al., MSR 2022). It contains issues from the Jira trackers of 39 open-source agile projects of 12 organisations (such as Apache, Atlassian and MongoDB). It is shared as one large MySQL database file (637 MB zipped, 4.3 GB unzipped). Instead of installing MySQL, we wrote a Python script that reads the file directly and extracts only the tables we need (issues and projects). All 39 projects have story points, and they are exactly the 39 projects used in the guideline's project-pair list.

Cleaning. We removed issues that are not useful for learning effort:

- Issues that were not completed (e.g. *Won't Fix*, *Duplicate*, still open), because their story points do not reflect real work.

- Issues with 0 story points, or more than 100 (clear data-entry mistakes; the largest value in the raw data was 26,710,114).
- Exact duplicates.

This left **51,777 issues** out of 65,427. The projects are very different in size, from 103 issues (Java driver) to 17,894 issues (Lsstcorp Data Management). They also use different story point scales: the typical issue is worth 1 point in some projects and 5–8 points in others.

Text. For each issue we used the title plus the description. TAWOS also provides a version of the description without code snippets, which we used. We removed links and Jira formatting, and split the description into sentences. An issue has 6.8 sentences on average.

Note on project names. One name in the guideline pair list, "C++ driver (Appcelerator)", does not exist in TAWOS. The only TAWOS project left unmatched is *Aptana Studio*, which belongs to the Appcelerator group of projects, so we used it for this name.

4. Methodology

4.1 Overall pipeline

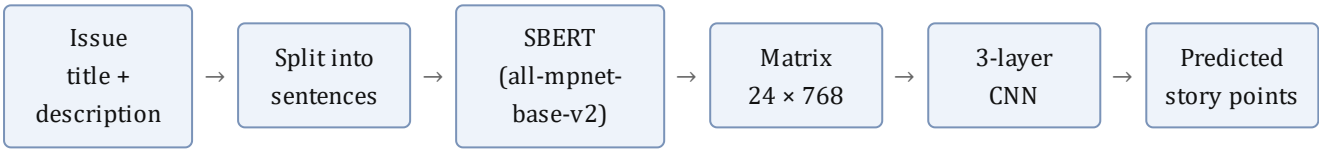


Figure 1: Steps from issue text to a story point prediction.

4.2 Text representation with SBERT

Every issue is turned into a list of sentences: the title first, then the description sentences in order. We keep at most 24 sentences, which covers 97% of the issues. Each sentence goes through the SBERT model, which gives a 768-number vector. So every issue becomes a **24 × 768 matrix**; shorter issues are padded with zeros, and a mask tells the model which rows are real. The SBERT model itself is not trained further: we only use it to produce the vectors. Encoding all 51,777 issues took about 6 minutes on our laptop GPU (NVIDIA RTX 4080).

4.3 The three-layer CNN

A 1-D convolutional neural network (CNN) reads the matrix. Each filter looks at **three neighbouring sentences at a time**, so it can use information that is spread over a few consecutive sentences rather than just one.

Layer	Details	Output
Input	SBERT vectors of up to 24 sentences	24 × 768
Convolution 1	256 filters, window of 3 sentences, ReLU, dropout	24 × 256
Convolution 2	128 filters, window of 3 sentences, ReLU, dropout	24 × 128

Layer	Details	Output
Convolution 3	64 filters, window of 3 sentences, ReLU	24 × 64
Pooling	max-pooling and average-pooling over the sentences, joined together	128
Output	fully connected layer (64 units, ReLU) followed by a single output neuron	1 story point value

Table 1: CNN architecture (about 0.72 million trainable parameters).

4.4 How we split the data

Target project (sorted by creation date, oldest → newest)

earlier 80% — not used	**TEST (latest 20%)**

Source project = training data (sorted by date), split into 5 folds for validation

Fold 1	Fold 2	**Fold 3 = validation**	Fold 4	Fold 5

Figure 2: Data split for one project pair (shown for the run where fold 3 is the validation fold).

- **Test set:** the **latest 20%** of the target project's issues (an 80:20 chronological split). The model never sees these issues during training or while we choose settings. The older 80% of the target project is not used at all, so the model knows nothing about the target project.
- **Training data:** all issues of the source project, sorted by date and cut into **5 folds** of equal size. In five runs, four folds are used for training and one for validation, so every fold is the validation fold once.
- **Choosing the settings:** we tried four settings (learning rate 0.001 or 0.0003, dropout 0.2 or 0.5) and picked the one with the lowest average validation MAE over the five folds.
- **Final model:** trained on all the source data with the chosen setting, then tested once on the test set. We repeated this with three random seeds (42, 43, 44) and report the average.

4.5 Training settings

Setting	Value
Loss function	L1 loss (this is the same as the MAE, so the model directly minimises MAE)
Optimiser	AdamW, weight decay 0.0001, batch size 64
Learning rate / dropout	chosen per pair from {0.001, 0.0003} × {0.2, 0.5}
Epochs	up to 60, early stopping after 8 epochs without improvement on the validation fold
Random seeds	42 (validation), and 42, 43, 44 (final models)
Software / hardware	Python 3.13, PyTorch 2.11, sentence-transformers 6.2; NVIDIA RTX 4080 Laptop GPU

Table 2: Training settings. All of them are stored in `configs/config.yaml` in our repository.

The complete experiment (39 pairs × 4 settings × 5 folds, plus the final models) took about **17 minutes** on the GPU.

5. Results

5.1 Five-fold validation results

Validation is done on the source (training) project. Table 3 gives the average over the 39 pairs; the results for each pair are in Appendix A.

Model	Validation MAE (mean ± std over 39 pairs)	Validation MdAE (mean ± std over 39 pairs)
SBERT + CNN	1.82 ± 1.52	1.19 ± 0.83
Median baseline	2.00	–

Table 3: Five-fold validation results on the training projects.

Inside the training project, the model is better than the median baseline in **34 of the 39 pairs**. Figure 3 shows the five fold scores for every pair. For most projects the folds are close to each other. A few projects with some very large story points (e.g. pair 11, Moodle) vary a lot.

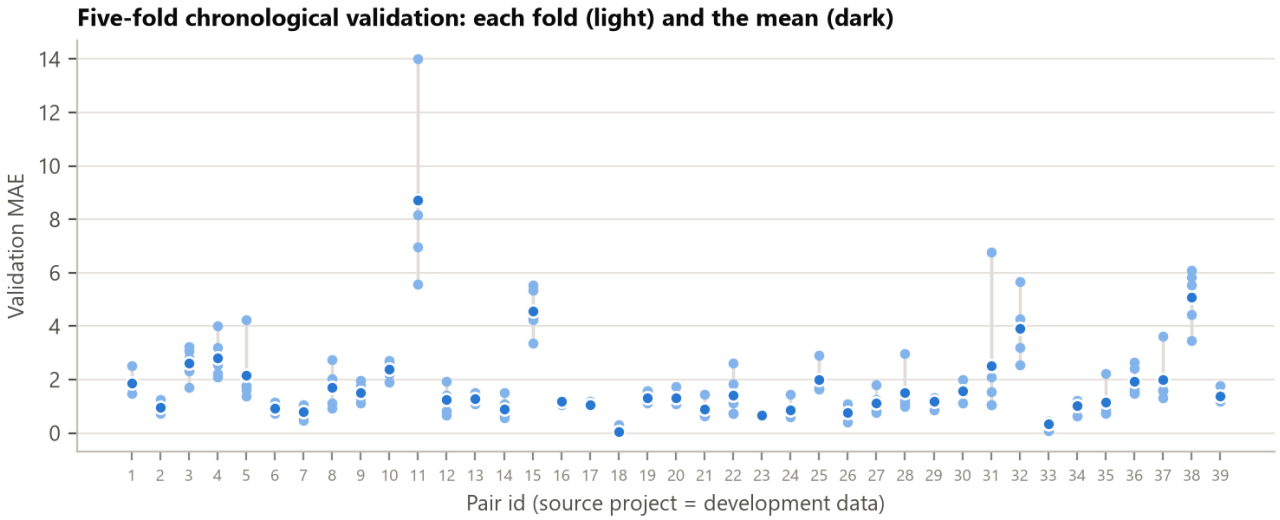


Figure 3: Validation MAE of each of the five folds (light dots) and their average (dark dot) for every pair.

5.2 Final test results

Table 4 is the main result in the format asked for in the guidelines. It shows the average over the 39 test sets (the latest 20% of each target project).

Representation	Model configuration	MAE	MdAE
SBERT sentence embeddings	all-mpnet-base-v2 (768-d, not fine-tuned), up to 24 sentences per issue → 3-layer 1-D CNN, L1 loss, AdamW; learning rate and dropout chosen by 5-fold validation; average of 3 seeds	**2.21**	**1.85**
No text (baseline)	always predict the median story point of the source project	2.16	1.88
No text (baseline)	always predict the mean story point of the source project	2.56	2.19

Table 4: Held-out test results (average of the 39 project pairs; lower is better).

Our model has the **lowest MdAE** and is better on average than the mean baseline. Its MAE, however, is about the same as the median baseline: it is better in 15 pairs and worse in 24. We checked this with a Wilcoxon signed-rank test, which found no significant difference ($p = 0.59$). The results were stable across the three random seeds: the test MAE changed by only 0.05 story points on average.

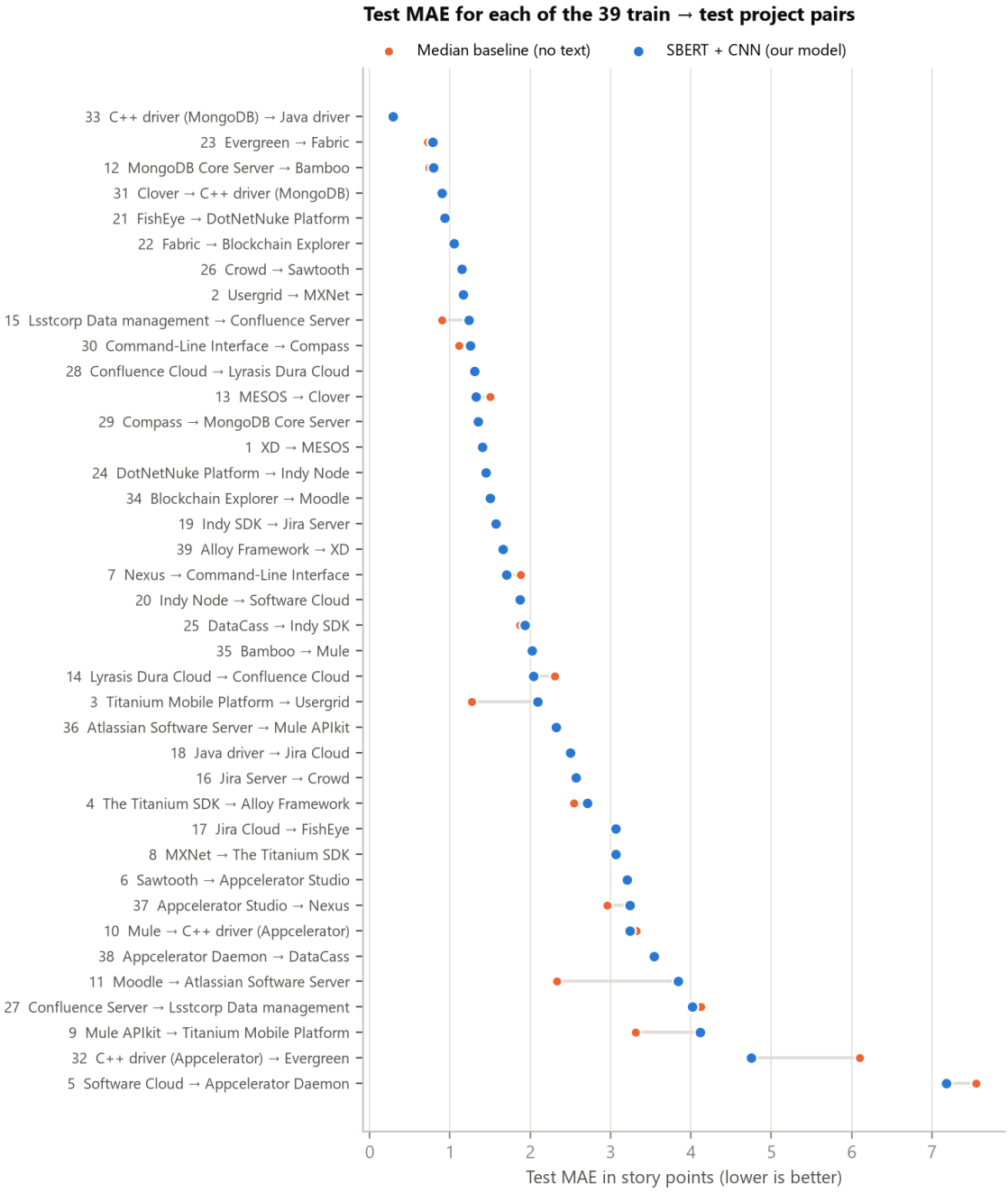


Figure 4: Test MAE of our model (blue) and the median baseline (orange) for every project pair. Where only blue is visible, both are almost equal.

Figure 4 shows that the error depends much more on *which pair* is tested than on the model. The best pair is C++ driver (MongoDB) → Java driver (MAE 0.29), where almost every issue is worth 1 point. The worst is Software Cloud → Appcelerator Daemon (MAE 7.18): the training project's typical issue is 3 points, but the test issues are usually 8 points.

5.3 Training project vs. new project

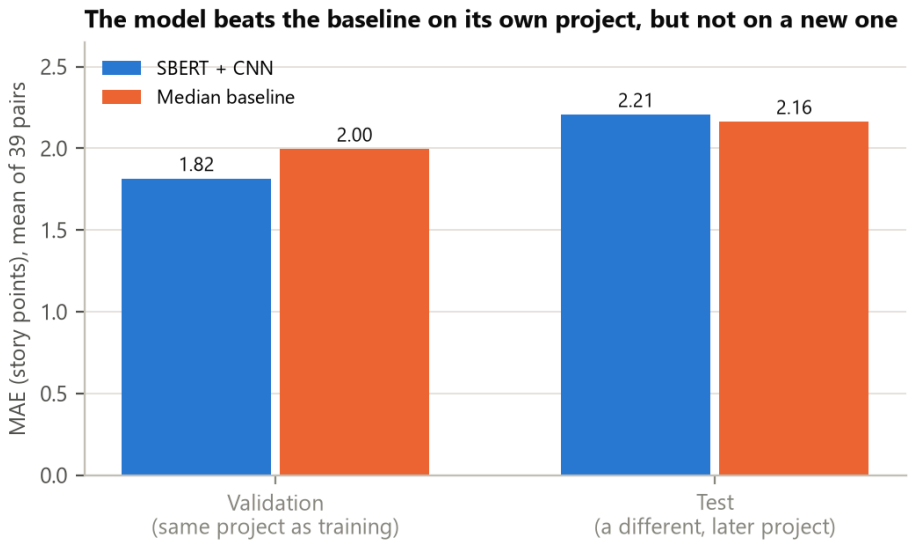


Figure 5: Average MAE of our model and the median baseline on the training project (validation) and on the new project (test).

Figure 5 summarises our most important observation. On the project it was trained on, the model is about 7% better than the median baseline. On a new project this advantage disappears.

5.4 Extra checks

- **Validation that always looks forward in time.** In our five-fold validation, the model is sometimes trained on issues that are newer than the validation issues. To check that this did not affect our choices, we also ran a "forward-chaining" validation, where the model is always validated on newer issues. The average validation MAE was almost the same (1.76 vs 1.82).
- **Training only on issues older than the test issues.** We also tried using only those source issues that were created before the target's test issues. This is possible for only 29 of the 39 pairs; for the other 10, the source project is newer than the target. The model again performed the same as the median baseline (MAE 2.42 vs 2.42).

6. Discussion

6.1 Which representation achieves the lowest MAE and MdAE?

Our problem statement only covers SBERT sentence embeddings, so we compared SBERT + CNN with the two baselines.

- **Lowest MdAE: SBERT sentence embeddings + CNN** (1.85). For a typical issue, our model's error is the smallest.
- **Lowest MAE: the median baseline**, by a very small and not significant margin (2.16 vs 2.21). If all 10,354 test issues are counted together instead of averaging per pair, our model has the lowest MAE (3.00 vs 3.11).

We saved the exact train/test split of every pair in our repository (`results/splits/`). The groups working on BERT word embeddings (PS-4) and combined embeddings (PS-12) for the same setting can therefore compare with us on exactly the same test issues.

6.2 Possible explanations for the results

1. **Every team has its own story point scale.** “3 points” in one project can mean the same work as “8 points” in another. The bigger the difference between the typical story point of the training and the test project, the bigger the error (correlation r = 0.76). A model that only reads text cannot know the scale of a new team.
2. **The text says only a little about effort.** Even inside one project, the model is only a little better than the median. When the signal is this weak, the safest prediction for MAE is close to the median. In about half of the pairs, the model learned to predict almost the same value for every issue.
3. **SBERT captures meaning, not effort.** SBERT was trained to find sentences with similar meaning. Two issues can be about very similar topics but need very different amounts of work, and we did not fine-tune SBERT for effort.
4. **Projects change over time.** The test issues are the newest ones. Teams change how they estimate over the years, which makes the newest issues harder to predict.

6.3 Limitations and threats to validity

- **Story points are subjective.** They depend on each team, so comparing MAE between projects is not completely fair. We report results for every pair as well as the average.
- **Small search for settings.** Because of limited time and compute, we tried only four settings. A larger search, or fine-tuning SBERT, might give better results.
- **Small test sets.** Some test sets are very small (as few as 21 issues), so the results for those pairs are not very reliable.
- **Validation folds.** In four of the five folds the model is trained on some newer issues. Our forward-chaining check (Section 5.4) showed that this did not change the results much.
- **Our choices.** The cleaning rules and the mapping of “C++ driver (Appcelerator)” to Aptana Studio are our own decisions. Different choices could change the numbers a little.
- **Generalisation.** We used only open-source projects with English text and one training project per test project. The results may be different for company projects, or when training on several projects together.

7. Conclusion and what we learned

We built a complete pipeline that predicts story points from issue text using SBERT sentence embeddings and a three-layer CNN, and we evaluated it on 39 cross-project pairs with chronological splits. The model reaches a test MAE of 2.21 and MdAE of 1.85 story points. That is about the same as always predicting the training project's median, but with a slightly lower median error. The model does learn useful information from the text inside its own project. When it moves to a new project, the difference in story point scales between teams cancels out this advantage.

What we learned from this project:

- Always compare with a simple baseline. Our model looked good on its own, but the median baseline showed that most of the work was done by predicting typical values.
- How the data is split matters a lot. Chronological, cross-project testing is much harder (and more honest) than a random split inside one project.
- A good validation score on one project does not guarantee good results on another project.
- Real datasets need careful cleaning. We found story points as large as 26 million.

Future work: use a few labelled issues of the new project to adjust the story point scale; train on several source projects together; fine-tune SBERT for effort estimation; and combine sentence embeddings with BERT word embeddings.

8. How to run the code

All code, settings and results are in our GitHub repository (link on the first page). After installing the requirements (see `README.md`), the full experiment can be repeated with one command, and the demo predicts story points for any text:

```
.\run_all.ps1          # full experiment, about 40 minutes on a GPU

python demo.py --tag demo --pair 1 --title "Fix login failure" \
       --description "The login fails when a user enters an incorrect password."
```

References

1. V. Tawosi, A. Al-Subaihin, R. Moussa, F. Sarro. A Versatile Dataset of Agile Open Source Software Projects. MSR 2022.
2. N. Reimers, I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
3. M. Choetkiertikul et al. A Deep Learning Model for Estimating Story Points. IEEE Transactions on Software Engineering, 2019.
4. V. Tawosi, R. Moussa, F. Sarro. Deep Learning for Agile Effort Estimation: Have We Solved the Problem Yet? IEEE Transactions on Software Engineering, 2023.
5. Y. Kim. Convolutional Neural Networks for Sentence Classification. EMNLP 2014.

Appendix A: Results for every project pair

Train issues = issues of the source project. Test issues = latest 20% of the target project. 5-fold val. MAE = mean ± standard deviation over the five validation folds. Test values are the average of the three seeds.

#	Train project → Test project	Train issues	Test issues	5-fold val. MAE	Test MAE	Test MdAE	Median baseline MAE
1	XD → MESOS	2408	527	1.86 ± 0.35	1.40	0.99	1.41
2	Usergrid → MXNet	336	25	0.95 ± 0.18	1.16	0.99	1.16
3	Titanium Mobile Platform → Usergrid	1165	67	2.63 ± 0.56	2.09	1.55	1.27
4	The Titanium SDK → Alloy Framework	3512	48	2.81 ± 0.71	2.71	2.49	2.54
5	Software Cloud → Appcelerator Daemon	233	42	2.15 ± 1.06	7.18	4.59	7.55
6	Sawtooth → Appcelerator Studio	831	536	0.94 ± 0.14	3.20	3.02	3.18
7	Nexus → Command-Line Interface	1395	59	0.80 ± 0.23	1.70	1.01	1.88
8	MXNet → The Titanium SDK	123	702	1.72 ± 0.66	3.07	2.01	3.06
9	Mule APIkit → Titanium Mobile Platform	405	233	1.53 ± 0.31	4.11	2.70	3.31
10	Mule → C++ driver (Appcelerator)	3041	105	2.40 ± 0.30	3.24	2.18	3.31
11	Moodle → Atlassian Software Server	1246	49	8.70 ± 2.87	3.84	2.96	2.33
12	MongoDB Core Server → Bamboo	599	91	1.25 ± 0.45	0.80	0.78	0.74
13	MESOS → Clover	2636	45	1.28 ± 0.16	1.32	1.05	1.50
14	Lyrasis Dura Cloud → Confluence Cloud	564	33	0.91 ± 0.35	2.03	1.77	2.30
15	Lsstcorp Data management → Confluence Server	17894	96	4.56 ± 0.79	1.24	0.97	0.90
16	Jira Server → Crowd	275	34	1.17 ± 0.07	2.57	2.99	2.57
17	Jira Cloud → FishEye	261	36	1.07 ± 0.07	3.06	3.49	3.07
18	Java driver → Jira Cloud	103	52	0.06 ± 0.12	2.50	3.00	2.50
19	Indy SDK → Jira Server	632	55	1.33 ± 0.21	1.57	1.16	1.60
20	Indy Node → Software Cloud	547	47	1.32 ± 0.22	1.87	1.01	1.87
21	FishEye → DotNetNuke Platform	182	375	0.90 ± 0.30	0.93	1.00	0.93
22	Fabric → Blockchain Explorer	550	73	1.40 ± 0.73	1.05	0.69	1.08
23	Evergreen → Fabric	4174	110	0.67 ± 0.04	0.78	0.95	0.72
24	DotNetNuke Platform → Indy Node	1876	109	0.88 ± 0.29	1.45	1.00	1.45
25	DataCass → Indy SDK	161	126	2.02 ± 0.47	1.93	1.76	1.87
26	Crowd → Sawtooth	172	166	0.77 ± 0.22	1.15	0.98	1.14
27	Confluence Server → Lsstcorp Data management	479	3579	1.12 ± 0.38	4.02	1.54	4.12
28	Confluence Cloud → Lyrasis Dura Cloud	167	113	1.53 ± 0.74	1.30	0.99	1.31
29	Compass → MongoDB Core Server	401	120	1.18 ± 0.17	1.35	2.00	1.35

#	Train project → Test project	Train issues	Test issues	5-fold val. MAE	Test MAE	Test MdAE	Median baseline MAE
30	Command-Line Interface → Compass	294	80	1.57 ± 0.28	1.25	1.01	1.11
31	Clover → C++ driver (MongoDB)	224	34	2.51 ± 2.17	0.90	1.00	0.90
32	C++ driver (Appcelerator) → Evergreen	525	835	3.92 ± 1.06	4.75	5.00	6.10
33	C++ driver (MongoDB) → Java driver	170	21	0.34 ± 0.13	0.29	0.00	0.29
34	Blockchain Explorer → Moodle	367	249	1.02 ± 0.21	1.50	1.96	1.51
35	Bamboo → Mule	454	608	1.16 ± 0.56	2.02	1.01	2.01
36	Atlassian Software Server → Mule APIkit	246	81	1.94 ± 0.49	2.32	2.00	2.32
37	Appcelerator Studio → Nexus	2681	279	2.00 ± 0.83	3.24	3.13	2.96
38	Appcelerator Daemon → DataCass	210	32	5.07 ± 0.98	3.54	3.51	3.53
39	Alloy Framework → XD	238	482	1.38 ± 0.21	1.66	2.00	1.66